Scaleform GFx

Scaleform 4.0 渲染器线程指南

本文介绍 Scaleform 4.0 中的多线程渲染配置。

作者： Michael Antonov
版本： 1.01
上次编辑时间： 2011 年 4 月 14 日

Scaleform

Copyright Notice

Autodesk® Scaleform® 4

© 2011 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and/or other countries: 3DEC (design/logo), 3December, 3December.com, 3ds Max, Algor, Alias, Alias (swirl design/logo), AliasStudio, Alias|Wavefront (design/logo), ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSnap, AutoSketch, AutoTrack, Backburner, Backdraft, Beast, Built with ObjectARX (logo), Burn, Buzzsaw, CAiCE, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DXF, Ecotect, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, HumanIK, IDEA Server, i-drop, Illuminate Labs AB (design/logo), ImageModeler, iMOUT, Incinerator, Inferno, Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, LiquidLight, LiquidLight (design/logo), Lustre, MatchMover, Maya, Mechanical Desktop, Moldflow, Moldflow Plastics Advisers, MPI, Moldflow Plastics Insight, Moldflow Plastics Xpert, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, PolarSnap, PortfolioWall, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Softimage|XSI (design/logo), Sparks, SteeringWheels, Stitcher, Stone, StormNET, StudioTools, ToolClip, Topobase, Toxik, TrustedDWG, U-Vis, ViewCube, Visual, Visual LISP, Volo, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. “AS IS.” AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	Scaleform 4.0 Renderer Threading Guide
Address	Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

目录

1 引言 1

2 多线程渲染器更改 1

2.1 渲染器创建 1

2.2 渲染缓存配置 2

2.3 纹理资源创建 3

2.4 渲染循环 5

2.5 电影关闭 6

3 多线程渲染概念 7

3.1 电影快照 7

4 渲染器设计 9

1 引言

Scaleform 4.0 包括一项新的多线程渲染器设计，它可实现在不同线程上同时执行电影推进 (Advance) 和渲染（显示）逻辑，从而提高整体性能并高效地将 Scaleform SDK 集成到多线程应用程序和游戏引擎之中。本文介绍这一新的渲染器设计的结构，概述为实现多线程渲染而在 Scaleform 4.0 中对 API 进行的更改，并提供在线程之间安全地达到 Scaleform 渲染所必需的示例代码。

2 多线程渲染器更改

许多 API 从 Scaleform 3.x 更改为 Scaleform 4.0，以利用新的设计和多线程渲染。此列表突出显示被更改的主要渲染器方面，它们是使用多线程技术时需要考虑的方面。

1. ***渲染器创建。***渲染器创建现在与渲染线程关联，而且还包括创建与平台无关的 Renderer2D 层。
2. ***渲染缓存配置。***现在，渲染器上配置了网格缓存和字体缓存，而未在 GFxLoader 上配置。
3. ***纹理资源创建。***纹理创建需要线程安全，或者由渲染线程提供服务。
4. ***渲染循环。***对电影进行的渲染现在是通过将 MovieDisplayHandle 传递给渲染线程然后在那里通过 Renderer2D::Display 对其进行渲染来完成的。
5. ***电影关闭。***GFx::Movie 发布操作可能需要由渲染线程来提供服务。
6. ***自定义渲染器。***重新设计了与平台无关的 API，以便为硬件顶点缓冲区缓存和分批处理提供支持。GRenderer 在 Scaleform 3.X 中的角色现在由 Render::HAL 类进行处理。

2.1 渲染器创建

Scaleform 端渲染器初始化涉及创建两个类：Render::Renderer2D 和 Render::HAL。Renderer2D 是一种矢量图形渲染器实现，提供渲染电影对象时使用的 Display 方法。Render::HAL 是 Renderer2D 使用的一个“硬件抽象层”接口；其特定实现位于特定平台的命名空间，例如 Render::D3D9::HAL 和 Render::PS3::HAL。这两个命名空间都是通过与下述情况相似的逻辑创建的：

```
D3DPRESENT_PARAMETERS   PresentParams;
IDirect3DDevice9*       pDevice = ...;
ThreadId                renderThreadId = Scaleform::GetCurrentThreadId();
Render::ThreadCommandQueue *commandQueue = ...;
Ptr<Render::D3D9::HAL>  pHal;
Ptr<Render::Renderer2D> pRenderer;

pHal = *SF_NEW Render::D3D9::HAL(commandQueue);
```

```
if (!pHal-> InitHAL(Render::D3D9::HALInitParams(pDevice, PresentParams, 0,
                                                renderThreadId)))
{
   return false;
}
pRenderer = *SF_NEW Render::Renderer2D(pHal);
```

在此案例中，HALInitParams class 提供针对个别平台的初始化参数。在这当中， pDevice 与 PresentParams 提供予由用户设定的硬件的相应信息、而 renderThreadId 识别渲染器使用的渲染线程。ThreadCommandQueue 是渲染线程中可能需要实现的一个接口实例，本文将对此进行详细介绍。

Renderer2D 和 HAL 类并非线程安全的，而是设计为仅在渲染线程上使用。它们通常是由于初始化请求而在渲染线程上创建的，或者在渲染线程受阻时在主线程上创建。

2.2 渲染缓存配置

在 Scaleform 4.0 中，网格缓存和字形缓存都与 Renderer2D 关联，而且是在渲染线程上维护的。此行为不同于 Scaleform 3.x，因为在那里，这些状态是在 GFxLoader 上配置的。缓存是在通过调用 InitHAL 来配置关联的 Render::HAL 时创建的；而在调用 ShutdownHAL 时释放其内存。

网格缓存存储顶点和索引缓冲区数据，而且通常直接从视频内存直接分配；它是由系统特定的逻辑，作为 Render::HAL 的组成部分进行管理的。网格缓存内存以大块分配，HAL 初始化时预分配第一块，其后可以按一个固定限额增加。网格缓存配置如下：

```
MeshCacheParams mcp;
mcp.MemReserve          = 1024 * 1024 * 3;
mcp.MemLimit            = 1024 * 1024 * 12;
mcp.MemGranularity      = 1024 * 1024 * 3;
mcp.LRUTailSize         = 1024 * 1024 * 4;
mcp.StagingBufferSize   = 1024 * 1024 * 2;

pRenderer->GetMeshCacheConfig()->SetParams(mcp);
```

在这里，MemReserve 定义分配的缓存内存的初始数量，而 MemLimit 定义最大可分配数量。如果这两个值相同，初始化后就不会发生任何缓存分配。MeshCache 按块增大（由 MemGranularity 指定），一旦存在的 LRU 缓存内容大于 LRUTailSize 所规定的，MeshCache 就会缩小。StagingBufferSize 是一个用于构建批量的系统内存缓冲区；它可以在控制台上设置为 64K（那里的默认值），但在 PC 上必须大于此值，因为它还充当一个二级缓存。

字形缓存用于栅格化字体，以便于高效地自纹理绘制字体。字形缓存动态更新，并具有一个初始化时确定的固定大小。字形缓存配置如下：

```
GlyphCacheParams gcp;
  gcp.TextureWidth  = 1024;
  gcp.TextureHeight = 1024;
  gcp.NumTextures   = 1;
  gcp.MaxSlotHeight = 48;

  pRenderer->GetGlyphCacheConfig()->SetParams(gcp);
```

用于字形缓存的配置设置实际上与 Scaleform 3.3 相同。TextureWidth、TextureHeight 和 NumTextures 定义将要分配的仅字形纹理的大小和数量。MaxSlotHeight 指定将为单个字形分配的最大槽高（单位：像素）。

对于要修改的初始设置，调用 InitHAL 之前，两个缓存均应配置。如果在 InitHAL 之后调用 SetParams，关联的缓存会被刷新，而且内存会被重新分配。

2.3 纹理资源创建

Scaleform 4.0 包括一个重新设计的 GFx::ImageCreator 以及一个新的 Render::TextureManager 类，它们允许从视频内存直接加载图像。与 Scaleform 的早期版本相似，ImageCreator 是安装在 GFx::Loader 上的状态对象，用来自定义图像数据加载，具体情况如下：

```
  GFx::Loader loader;
  ...

  SF::Ptr<GFx::ImageCreator> pimageCreator =
      *new GFx::ImageCreator(pRenderThread->GetTextureManager());
  loader.SetImageCreator(pimageCreator);
```

默认 ImageCreator 取构造函数中的一个可选 TextureManager 指针；如果指定了一个纹理管理器且该纹理管理器不会丢失数据，就会将图像内容直接加载到纹理内存之中。TextureManager 对象是由 Render::HAL::GetTextureManager() 方法返回的，后者应在渲染线程上调用。上面的代码示例假定一个渲染线程类提供 GetTextureManager 访问器方法，该方法只能在初始化渲染后调用。此时，此相关性表明：如果要将内容直接加载到纹理之中，在加载内容之前需要初始化渲染。如果未指定纹理管理器，就会先将图像数据加载到系统内存中，因而可在初始化渲染器之前发生。

与 Render::HAL 的渲染方法不同的是，TextureManager 方法（如 CreateTexture）必须是线程安全的，因为它们可以从不同加载线程调用。此线程安全通过以下两种方法之一实现：

- 通过线程安全的方式（例如在控制台上）实现纹理内存分配，或者在用 D3DCREATE_MULTITHREADED 旗标包装设备时利用 D3D9 实现。
- 通过借助 ThreadCommandQueue 接口（在 Render::HAL 构造函数中指定）将纹理创建指派给渲染线程。

此第二种方法意味着，***假如用非多线程 D3D 设备进行多线程渲染，就必须实现 ThreadCommandQueue*** 并根据纹理创建需要为其提供服务。若非如此，一旦从第二个线程调用，纹理创建就会阻滞。请参阅 Platform::RenderHALThread 类，了解示例实现情况。

2.4 渲染循环

通过多线程渲染，传统渲染循环被拆分成两个部分：推进线程执行的推进/输入处理逻辑，以及执行绘图命令（如绘制帧摄）的渲染线程循环。Scaleform 4.0 APIs 通过定义可安全地传递到渲染线程的 GFx::MovieDisplayHandle 来实现多线程技术。GFx::Movie 对象本身并不应传递到渲染线程，因为它本质上并非线程安全的，而且不再包含 Display 方法。

一般电影渲染过程可分为以下几个步骤：

1. ***初始化。***由 GFx::MovieDef::CreateInstance 创建 GFx::Movie 之后，用户通过设置视窗、获取显示句柄并将其传递到渲染线程来对 GFx::Movie 进行配置。
2. ***主线程处理循环。***在每个帧上，用户处理输入并调用 Advance 来执行时间线和 ActionScript 处理。在对电影进行一切 Invoke/DirectAccess API 修改之后调用 Advance 是最后一次调用非常重要，因为那是对帧拍摄现场快照的地方。在 Advance 之后，电影将一个 Scaleform 绘制帧请求提交给渲染线程。
3. ***渲染。***一旦渲染线程收到绘制帧请求，它就抓取摄 MovieDisplayHandle 的最新捕获的快照，并在屏幕上对其进行渲染。

以下参考将详细说明这些步骤。

```
    //------------------------------------------------------------------------
// 1. 电影初始化

    Ptr<GFx::Movie>          pMovie = ...;
    GFx::MovieDisplayHandle  hMovieDisplay;

// 创建电影后配置视窗并抓取
// 显示句柄。
    pMovie->SetViewport(width, height, 0,0, width, height);
    hMovieDisplay = pMovie->GetDisplayHandle();

// 将该句柄传递到渲染线程；这是引擎特定的。在 Scaleform Player 中，
// 只是通过将一个内部函数调用排进队列里来完成的。
    pRenderThread->AddDisplayHandle(hMovieDisplay);


    //------------------------------------------------------------------------
// 2. 处理循环

// 在主线程上处理输入和处理操作。电影回调，
// 如 ExternalInterface，也在这里从 Advance 进行调用。
    float deltaT = ...;
    pMovie->HandleEvent(...);
    pMovie->Advance(deltaT);
```

```
// 等待前一个帧渲染完成并将
// 绘制帧请求排入队列。
    pRenderThread->WaitForOutstandingDrawFrame();
    pRenderThread->DrawFrame();


    //-------------------------------------------------------------------------
// 3. 渲染 – 渲染线程 DrawFrame 逻辑
    Ptr<Render::Renderer2D> pRenderer = ...;

    pRenderer->BeginFrame();
    bool hasData = hMovieDisplay.NextCapture(pRenderer->GetContextNotify());
    if (hasData)
        pRenderer->Display(hMovieDisplay);
    pRenderer->EndFrame();
```

尽管该逻辑的大部分一目了然，但仍然需要注意以下几个细节：

- WaitForOutstandingFrame – 此帮助函数确保渲染线程不会超前一个帧以上。这不仅节约 CPU 处理时间，而且确保帧渲染线程排队命令不会超越与电影内容同步。在单线程模式中，这并不必要，因为可以直接在 Advance 之后执行绘制帧逻辑。
- MovieDisplay.NextCapture – 要想“抓取”Advance 捕获的最新快照，就必须进行此调用，从而使其保持最新，以便进行渲染。一旦电影不再可用，此函数就会返回 false（假），在这种情况下，不会绘制任何东西。

2.5 电影关闭

当与多线程渲染一起使用时，如果在毁坏渲染器对象之前执行 GFx::Movie Release 操作，则可能需要有渲染线程为其提供服务；要向电影的内部数据结构发布任何渲染线程参考，就必需此服务。默认情况下，将会通过前面提及的 Render::ThreadCommandQueue 接口从 Movie 解构函数对电影关闭命令进行排队。开发者必须实现 ThreadCommandQueue 接口以确保正确关闭。

作为在渲染线程上为命令排队提供服务的替代选择，推进线程可使用电影关闭轮询接口先发起关闭，然后等待其完成之后再发布电影。为此，请先调用 Movie::ShutdownRendering(false) 以发起关闭，然后使用 Movie::IsShutdownRenderingComplete 来针对每个帧测试其是否已经完成。在渲染线程上，NextCapture 将处理发起的关闭并返回 false。一旦处理关闭，推进线程就可无阻滞地执行电影发布。

3 多线程渲染概念

使用 Scaleform 进行的多线程渲染至少包含两个线程 – ***推进线程***和***渲染线程***。

推进线程一般是创建电影实例、处理其输入处理以及调用 GFx::Movie::Advance 以执行 ActionScript 和时间线动画的线程。应用程序中可以有不止一个 Advance 线程；不过，每部电影一般只由一个 Advance 线程访问。在多数 Scaleform 示例中，Advance 线程也是主要应用程序控制线程，根据需要向渲染线程发出命令。

渲染线程的主要任务是渲染屏幕上的图形，渲染线程通过向图形设备输出命令（一般经由像 Render::HAL 这样的应用层）来完成此操作。要最大限度地减少通信开销，Scaleform 渲染线程处理像添加电影摄和绘制帧摄这样的宏命令，而不是像绘制三角形摄这样的宏命令。渲染线程还管理网格缓存和字形缓存。

渲染线程可以锁步工作，或者独立于控制推进线程工作，具体情况如下：

- 利用锁步渲染，控制线程向渲染线程发出绘制帧摄命令，一般在电影推进处理之间。这两个线程并行工作，渲染线程绘制前一个帧，而推进线程则处理下一个帧。在多核系统上，这会提高总体性能，同时还保持线程框架之间的一对一关系。
- 对于独立工作模式，渲染线程在绘制帧时不受可能以不同帧速运行的推进线程影响。对电影的修改会在修改完成之后不久显示出来，这是渲染线程抽出时间绘制已更新的电影快照的下一个时间段。

Scaleform Player 应用程序和 Scaleform 示例利用锁步渲染模式，这一方面因为设置更方便，另一方面因为游戏中更为普遍。

3.1 电影快照

在多线程渲染期间，Advance 线程可在渲染线程出于显示目的访问内部电影状态的同时修改内部电影状态。由于两种线程对数据的非确定性访问会导致崩溃和/或坏帧，新的渲染器利用快照确保渲染线程始终看到一个连贯的帧。

在 Scaleform 中，电影快照是电影在给定时间点的一种被捕获状态。默认情况下，在调用 GFx::Movie::Advance 结束时自动捕获快照，也可以通过调用 GFx::Movie::Capture 来明确地捕获。一旦捕获一个快照，渲染线程就可以使用其帧状态，但与电影的动态状态分开存储。绘制帧时，渲染线程在一个电影句柄上调用 NextCapture 以抓取最近的快照并将其用于渲染。

此设计对于开发者来说具有多种含义：

- 当与渲染同时执行时，Movie::Advance 和输入处理逻辑并不需要一个关键节。
- 可以按照不同频率调用 Capture 和 NextCapture。如果进行多次连续的 Capture 调用，就会合并电影快照。如果没有足够的 Capture 调用，同一帧就可能会渲染多次。
- 调用 Capture 或 Advance 方法之前，渲染线程看不到由于输入、Invoke 或 Direct Access API 对 Movie 进行的修改。

最后一点说明 Scaleform 4.0 与早期版本之间的一个重要行为差异。尽管在 Scaleform 3.3 中开发者可以调用 Display 遵循的 Movie::Invoke 来呈现对屏幕的更改，但现在要呈现更改他们还必须调用 Capture 或 Advance。一般情况下，不需要这样做，因为可以在调用 Advance 之前把输入处理和 Invoke 调用组合在一起。

4 渲染器设计

在大体上了解多线程渲染之后，我们现在可以看看 Scaleform 4.0 渲染设计图表。

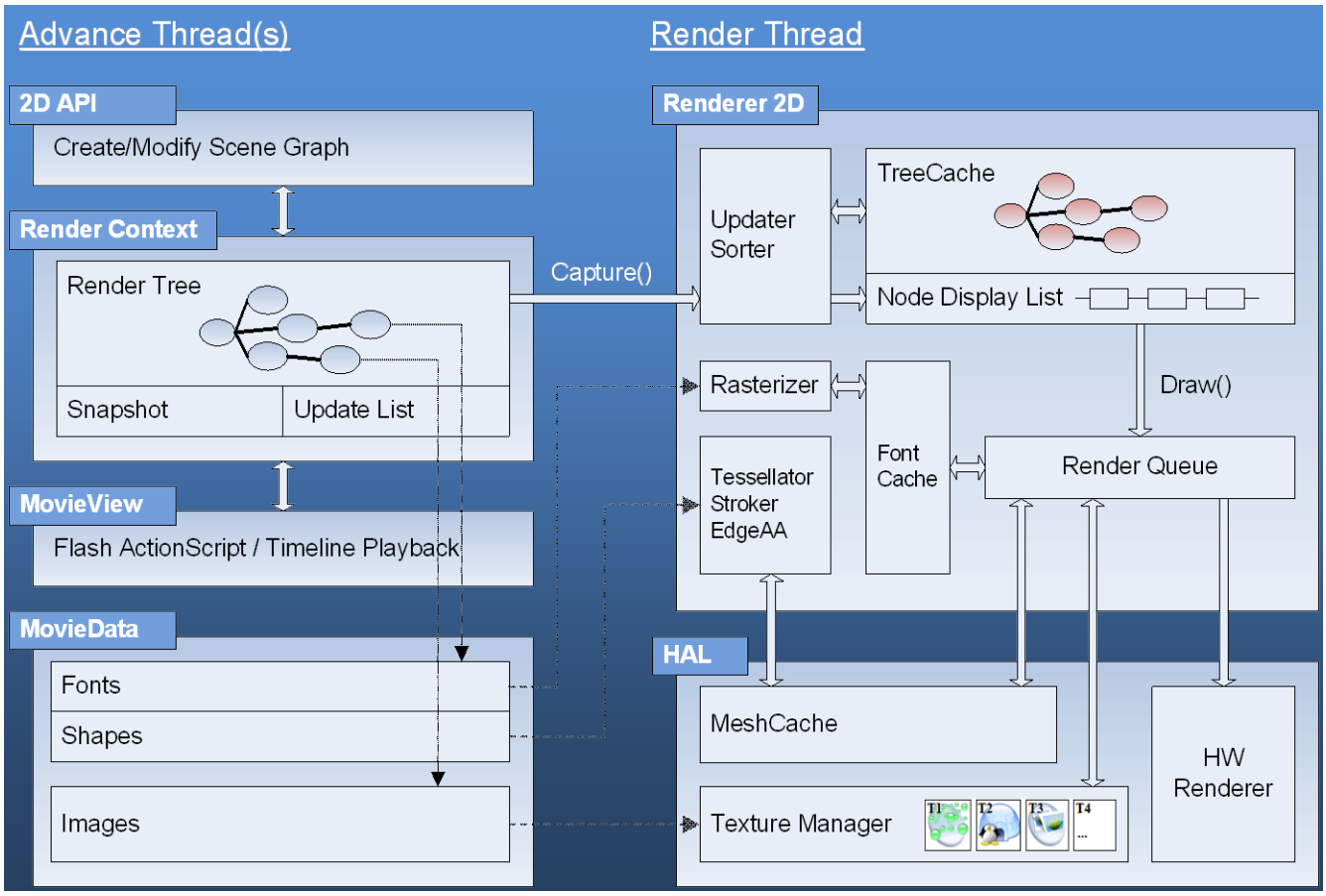


图 1：渲染设计

此图表显示与两个线程一起运行时渲染器子系统交互情况。在左边，Movie 和 MovieDef 对象分别代表 Scaleform 电影实例及其内部共享数据。Render Context 是管理 Render Tree 的渲染子系统的一个前端；它负责渲染树可访问的 2D 场景图表快照和更改列表。在 Scaleform 中，渲染上下文包含在 Movie 内，而且不需要最终用户访问。

在右边，渲染线程维护两个对象：Render::Renderer2D 和 Render::HAL。Renderer2D 是渲染引擎的核心；它包含 Display 方法，该方法用来将电影快照输出到屏幕。Render::HAL 代表硬件抽象层摂，并且是一个针对每个平台具有不同实现的抽象类。Render HAL 负责执行图形命令并管理像顶点和纹理缓冲区这样的资源。

从图表中可以看出，大部分渲染系统是在渲染线程上维护和执行的。具体说来，渲染线程负责所有下述任务：

- 维护渲染树的分类、缓存版本并在来自新快照的更改到达时对该版本进行更新。
- 维护 Font Cache 并在有新字形需要渲染时对 Font Cache 进行更新。
- 维护 Mesh Cache，其中包含细化的矢量数据的顶点和索引缓冲区。
- 管理一个 Render Queue，该 Render Queue 能够实现高效缓冲区更新/锁定管理，并在渲染时最大限度地减少 CPU 停机。
- 通过 HAL API 向设备发出图形命令。

在多数情况下，使缓存管理保持在渲染器线程上可提高性能，因为这可以将主线程解放出来处理其它工作密集型任务，例如，ActionScript 或游戏 AI。把缓存保持在渲染线程上还可以减少同步开销和内存使用，因为这消除了为在线程之间复制顶点和/或字形数据时留出额外缓冲区的必要。